S A M S U N G P R I S M

Industry-Academia Research Collaboration

Real-Time Predictive Texture Streaming

for Browser-Based XR Applications

Project Proposal & Implementation Plan

Project Track	Immersive Technologies / XR
Platform	Browser (Three.js + TF.js)
Prediction Model	Simplified DB-LSTM (PredRNNv3-inspired)
Duration	6 Weeks
Date	6 April 2026

1. Executive Summary

This document presents the full proposal and phased implementation plan for a real-time predictive texture streaming system developed as a Samsung PRISM research project. The system uses webcam-based eye-tracking to forecast where a user will look 100–200 milliseconds into the future, and proactively preloads high-resolution textures at that predicted location in a browser-rendered 3D scene — reducing texture pop-in and improving perceived visual quality without requiring any native application or device-specific SDK.

Core Problem	Traditional texture streaming systems react to the user's current field of view. On rapid head or eye movements, the reactive system cannot keep up — resulting in visible texture pop-in (low-res textures appearing briefly before the high-res version loads). This is especially disruptive in XR applications.
Proposed Solution	A gaze-predictive texture streamer: face-api.js tracks the user's eyes at 10 Hz; a lightweight dual-branch LSTM (inspired by PredRNNv3) predicts gaze position 200 ms ahead; a Three.js streaming manager preloads the correct mip level for objects at the predicted location before the user's gaze arrives there.

2. Background & Motivation

2.1 The Texture Pop-in Problem

Texture streaming divides high-resolution textures into multiple mipmap (mip) levels — low-resolution versions for objects far from the viewer's focus, and high-resolution for objects in the foveal zone. The streaming manager upgrades or downgrades mip levels based on the user's current gaze. The fundamental limitation of all reactive systems is latency: by the time the system detects that a high-res texture is needed, the user is already looking at the low-res placeholder. This lag — typically 150–400 ms — causes the pop-in artifact.

2.2 Why Prediction Helps

Human eye movement follows detectable short-term patterns: saccades have predictable velocity profiles; smooth pursuit movements follow object trajectories; reading follows left-to-right progressions. A model that learns these patterns can forecast the next gaze position with sufficient accuracy to preload textures before they are needed, eliminating the perceptual lag.

2.3 Why the Browser Platform

Running in the browser removes all platform dependencies — no Unity project setup, no Android Studio configuration, no headset required. Face-api.js provides production-quality 68-point facial landmark detection running in WebGL at 10 Hz. Three.js provides a full 3D rendering pipeline. TensorFlow.js enables on-device ML inference with GPU acceleration — all within a single HTML/JS codebase deployable with a single command.

3. Project Objectives

3.1 Primary Objectives

- Implement a real-time gaze tracking system at 10 Hz using webcam input and face-api.js landmark detection, extended with velocity computation and a structured data logger for ML training.
- Build a Three.js 3D scene with a multi-resolution texture streaming manager supporting three mip levels (64 px / 256 px / 512 px), establishing a reactive baseline.
- Implement a Kalman filter predictor operating on the gaze state vector [x, y, vx, vy] to forecast gaze position 200 ms ahead and drive proactive texture preloading.
- Design and train a lightweight dual-branch LSTM (Conv1D branch + self-attention branch + DBMM fusion gate, inspired by PredRNNv3) on collected gaze sequences; export to TensorFlow.js for in-browser inference.
- Quantify improvement across four metrics — pop-in events, load latency, GPU memory usage, and FPS stability — comparing reactive baseline vs. Kalman vs. DB-LSTM.

3.2 Stretch Objectives

- Fuse head orientation data (device gyroscope on mobile) with eye gaze as dual input streams, mirroring PredRNNv3's dual-branch memory architecture.
- Semantic texture prioritisation: classify scene objects by interaction type and preload interactive/UI textures before background geometry regardless of gaze distance.
- Adaptive prediction: use scene-type classification (static gallery vs. dynamic corridor) to select the best prediction model automatically.

4. Technical Architecture

4.1 System Pipeline

The system is structured as a four-stage pipeline. Stages 1–2 run in the gaze tracking interval (100 ms). Stages 3–4 run in the requestAnimationFrame render loop (~16 ms).

Stage	Name	Process	Output
1	Feature extraction	Webcam frame → face-api.js TinyFaceDetector → 68-point landmarks → left/right eye centroids	eyeX, eyeY in webcam pixel space
2	Gaze computation	Normalise to canvas space, mirror X, apply exponential smoothing, compute velocity	gazeX, gazeY, vx, vy; append to log
3	Prediction	Phase 1: identity (current pos). Phase 2: Kalman filter. Phase 3: DB-LSTM inference	predX, predY (200 ms ahead)
4	Streaming update	Project all objects to screen space, compute gaze distance, assign target mip, trigger async loads	Mesh materials updated; metrics recorded

4.2 Module Architecture

The codebase is split into six focused modules for Phase 1, with two additional modules added in Phases 2 and 3:

File / Module	Role	Key Functions
index.html	Application shell, toolbar, HUD, loading overlay	Scene canvas, webcam video element, mode/scene selectors
style.css	Dark industrial theme, HUD layout, overlay positioning	CSS variables, toolbar, gaze zone styling
gaze.js	Face-api.js tracking, velocity computation, data logger	init(), exportLog(), gazeupdate event, rolling buffer
streaming.js	Multi-resolution texture manager (reactive baseline)	register(), update(), pop-in counter, latency metrics
scene.js	Three.js scene setup, camera look, object placement	init(), loadScene(), render(), Gallery + Tunnel scenes
main.js	App entry point, RAF loop, overlay canvas, UI wiring	Bootstrap sequence, HUD updates, gaze circle overlay
[Phase 2] kalman.js	*Kalman filter predictor — 200 ms lookahead*	*predict(), update(), state vector [x,y,vx,vy]*
[Phase 3] model.js	*TF.js DB-LSTM inference wrapper*	*loadModel(), infer(), rolling input buffer*
[Phase 3] train.py	*Python training script for offline DB-LSTM*	*Conv1D branch, attention branch, DBMM gate, TF export*

4.3 Mip Resolution Tiers

Each scene object is associated with three texture versions, loaded on demand:

Tier	Resolution	Screen Distance Threshold	Simulated Load Latency	Visual Indicator
LOW	64 × 64 px	Always (default)	0 ms (pre-loaded)	Desaturated grey tint
MID	256 × 256 px	< 40% of screen radius	~80 ms	Warm neutral tint
HIGH	512 × 512 px	< 18% of screen radius (foveal)	~200 ms	Full vivid colour + label

5. Prediction Model — Simplified DB-LSTM

5.1 Inspiration: PredRNNv3

The prediction architecture is inspired by PredRNNv3 (Guo & Zhou, Neurocomputing 2026), which proposes a Dual-Branch LSTM (DB-LSTM) combining a CNN branch for local spatial features and a Vision Transformer branch for global spatial dependencies, fused via a Dual-Branch Modulation Module (DBMM). PredRNNv3 achieves state-of-the-art performance on spatiotemporal prediction benchmarks while significantly reducing parameter count compared to prior PredRNN variants.

5.2 Adaptation for Gaze Prediction

The full PredRNNv3 operates on 2D video frames with a Vision Transformer. For gaze prediction the input is a 1D sequence of gaze coordinates — a much simpler domain. The adaptation retains the dual-branch philosophy but scales down to ~50K parameters suitable for real-time TensorFlow.js inference in the browser:

Component	PredRNNv3 (original)	This project (adapted)
Input	2D video frames (H×W×C)	1D gaze sequence [x,y,vx,vy] × 10 ticks
CNN branch	3×3 Conv2D + channel attention on spatial maps	Conv1D over time axis — captures local motion patterns
Transformer branch	Multi-Head Window Self-Attention (ViT)	Scaled dot-product self-attention over 10-tick sequence
DBMM fusion	3×3 Conv + sigmoid gate + 1×1 Conv	Concat + sigmoid gate + linear layer
Output	Future video frames	Predicted (x,y) at t+2 (200 ms ahead)
Parameters	~6.2 M	~50 K (suitable for TF.js inference < 5 ms)
Training framework	PyTorch (GPU)	PyTorch offline → TF.js export for browser

5.3 Training Data

- Source: gaze log exported from the Phase 1 demo after 20–30 minutes of real user sessions.
- Format: sequences of [t, x, y, vx, vy] at 10 Hz, exported as structured JSON with canvas dimensions and smoothing parameters.
- Sample construction: sliding window of 10 ticks input → target is (x,y) at tick +2 (200 ms).
- Split: 80% train / 20% test. No data augmentation required given the simplicity of the input space.

5.4 Evaluation Strategy

The model is evaluated in three modes, compared head-to-head on the same gaze session:

- Reactive baseline: current gaze position drives streaming, no prediction.
- Kalman filter: linear constant-velocity predictor, 200 ms lookahead. Established baseline for comparison.
- DB-LSTM: the trained ML model. Should outperform Kalman on non-linear gaze motions (saccades, reading patterns).

6. Phase-by-Phase Implementation Plan

The project is structured across six phases over six weeks. Each phase has a clear milestone that must be demonstrated before the next phase begins. The first three phases establish the reactive foundation, the Kalman predictor, and the ML model. Phases 4–6 focus on evaluation, reporting, and stretch goals.

Phase 1 Gaze Logger + Three.js Scene + Reactive Baseline Week 1

Task	Detail / Output	Milestone
1.1 Gaze data logger	Extend trackFace() to compute velocity (vx, vy) per tick. Store { t, x, y, vx, vy } in rolling buffer (cap 36,000 frames = 1 hour). Add JSON export with metadata.	
1.2 Three.js scene	Replace flat video canvas with Three.js renderer. Build Gallery scene (8 textured panels in a ring) and Tunnel scene (corridor with side panels).	
1.3 Multi-res texture loader	Implement TextureStreamer with LOW/MID/HIGH mip tiers. Simulate loading latency (0 / 80 / 200 ms). Apply mip based on gaze distance from object.	
1.4 Reactive baseline verified	All four streaming metrics (pop-in count, load latency, GPU memory estimate, FPS) shown live in HUD overlay.	**✓ Reactive baseline working**

Phase 2 Kalman Filter Prediction — 200 ms Lookahead Week 2

Task	Detail / Output	Milestone
2.1 State vector	Define [x, y, vx, vy] state. Set transition matrix F (constant-velocity model). Tune process noise Q and measurement noise R using exported gaze logs.	
2.2 Predict + update loop	Kalman predict() on each trackFace() tick. update() with observed gazeX/Y. Extrapolate state 2 ticks forward for 200 ms lookahead.	
2.3 Wire to streaming	Replace current gazeX/Y with predX/predY as input to TextureStreamer.update(). Add A/B toggle in UI to switch reactive ↔ Kalman.	
2.4 Offline validation	Replay exported gaze logs through Kalman. Plot predicted vs actual. Confirm the filter leads the curve rather than lagging it.	**✓ Kalman prediction driving preloads**

Phase 3 Simplified DB-LSTM (PredRNNv3-inspired) — Browser ML Weeks 3–4

Task	Detail / Output	Milestone
3.1 Training data prep	Export 20–30 min gaze logs from Phase 1 demo. Build supervised dataset: input = last	

	10 ticks [x,y,vx,vy], output = (x,y) at t+2. 80/20 train/test split.	
3.2 Build model in Python	Conv1D branch (local motion patterns) + scaled dot-product attention branch (global sequence context) + sigmoid gate DBMM fusion. ~50K parameters.	
3.3 Train & evaluate vs Kalman	Train with AdamW, MSE loss. Compare prediction error (px) on held-out test set against Kalman baseline. Document results.	
3.4 Export to TF.js	Use tensorflowjs_converter to export saved model. Load in browser with tf.loadLayersModel(). Measure inference latency (target < 5 ms/tick).	
3.5 Live browser inference	Replace Kalman with DB-LSTM in prediction slot. Verify streaming manager receives valid predX/predY. All three modes selectable via UI toggle.	**✓ ML model live in browser**

Phase 4 Metrics Dashboard + A/B Evaluation Week 5		
Task	**Detail / Output**	**Milestone**
4.1 Instrument all metrics	Add pop-in counter, load latency timer, memory estimator to TextureStreamer. Track per-mode in separate buckets.	
4.2 A/B session runner	Add mode toggle (Reactive / Kalman / DB-LSTM) to UI. Run each mode for 2 min on same gaze session. Export results as JSON.	
4.3 On-screen dashboard	Extend HUD with live charts (sparkline or bar) showing pop-in and latency per mode. Clear visual comparison.	**✓ Quantified improvement demonstrated**

Phase 5 Report + Stretch Goals Week 6		
Task	**Detail / Output**	**Milestone**
5.1 Technical report	Document model architecture (reference PredRNNv3), quantitative results across all 4 metrics, ablation (reactive vs Kalman vs DB-LSTM), qualitative screen-recording analysis.	
5.2 Stretch: gaze + head fusion	Combine eye gaze with device gyroscope (orientation sensor on mobile) as two separate input streams — mirrors PredRNNv3 dual memory paths.	
5.3 Stretch: semantic priority	Classify objects by type (UI / interactive / background). Preload interactive textures regardless of gaze distance.	**✓ Final deliverable: demo + report**

7. Evaluation Metrics

All metrics are measured automatically by the streaming manager and exported as JSON for analysis:

Metric	Description	Compared Between
Pop-in events / min	Frames where required mip was unavailable on request	Reactive vs Predictive
Texture load latency (ms)	Time from texture needed to mip fully loaded	Reactive vs Predictive
GPU memory (KB)	Total loaded texture bytes estimated from pixel counts	All modes
FPS stability	Frame rate variance with prediction model running	With / without model
Prediction error (px)	Euclidean distance: predicted gaze vs actual gaze	Kalman vs DB-LSTM

8. Technology Stack

Layer	Technology	Purpose
Gaze tracking	face-api.js 0.22.2	TinyFaceDetector + 68-point landmarks; runs at 10 Hz in WebGL
3D rendering	Three.js r157	Scene, camera, materials, WebGL renderer, frustum culling
Camera controls	OrbitControls	Mouse-look fallback when webcam is unavailable
ML inference	TensorFlow.js 4.x	In-browser DB-LSTM inference with WebGL backend; < 5 ms/tick
ML training	PyTorch + AdamW	Offline training on collected gaze logs; exported via tf.js converter
Reference paper	PredRNNv3 (2026)	Dual-branch CNN+ViT LSTM architecture — inspires DB-LSTM design
Build / serve	No build step required	Plain HTML/JS/CSS; serve with python -m http.server 8080

9. Current Implementation Status

9.1 Completed (Phase 1 — Week 1)

- Gaze tracking module (gaze.js): face-api.js 10 Hz tracking, velocity computation, rolling log buffer with JSON export.
- Three.js scene (scene.js): Gallery and Tunnel scenes with 8–12 textured objects each; camera driven by gaze normalised coordinates.
- Multi-resolution texture streaming (streaming.js): three mip tiers with simulated loading latency; reactive baseline fully operational.
- Live HUD metrics overlay: FPS, detection rate, jitter, pop-in count, load latency, GPU memory estimate, log frame count.
- UI controls: scene selector, mode selector (Phase 2/3 slots pre-wired), export log button, webcam toggle.

9.2 Next Steps (Phase 2 — Week 2)

- Implement kalman.js: Kalman filter on state vector [x, y, vx, vy], 200 ms prediction lookahead.
- Validate offline using exported gaze JSON logs before wiring to live streaming.
- Enable Kalman mode in the UI mode selector dropdown.

9.3 Upcoming (Phases 3–6)

- Collect 20–30 minutes of gaze data using the Phase 1 export feature.
- Train simplified DB-LSTM in Python/PyTorch on the collected sequences.
- Export model to TF.js and integrate into the browser pipeline.
- Run A/B evaluation and produce the final quantitative comparison report.

Here's the full phase-by-phase breakdown in detail:

Phase 1 — Gaze Logger + Three.js Scene (Week 1)

Step 1.1 — Gaze data logger Extend the existing trackFace() function to push { t, x, y, vx, vy } into a rolling buffer every tick. Compute velocity as (gazeX - prevX) / dt. Add a download button that exports the buffer as JSON. Collect 20–30 minutes of real gaze data across different users and content — this becomes your training dataset for Phase 3.

Step 1.2 — Three.js scene Replace the flat video canvas with a Three.js scene. Start simple — a room with 5–6 textured planes/boxes visible. The gaze tracker from face-api.js will map to a 3D ray that intersects scene objects.

Step 1.3 — Multi-resolution texture loader For each object, prepare three versions of its texture (128px, 512px, 1024px). By default load the 128px version. When an object enters the current FOV, upgrade to 1024px. This is your **reactive baseline** — no prediction yet.

Milestone: Scene runs, textures load reactively, you can see pop-in when turning quickly.

Phase 2 — Kalman Filter Prediction (Week 2)

Step 2.1 — Implement Kalman filter in JS State vector is [x, y, vx, vy]. The transition matrix assumes constant velocity. Tune process noise Q and measurement noise R using your logged data. ~50 lines of plain JS, no libraries needed.

Step 2.2 — 200 ms lookahead After each predict() call, extrapolate the state vector 2 more ticks (at 10 Hz = 200 ms) to get predX, predY.

Step 2.3 — Wire to texture loader Pass predX, predY to the texture streaming manager instead of current gaze. Objects in the predicted FOV cone get their high-res mip preloaded 200 ms before the user looks there.

Step 2.4 — Validate offline Replay your logged gaze sequences through both the exponential smoother (current) and the Kalman filter. Plot predicted vs actual — you should see the Kalman filter lead the curve instead of lagging it.

Milestone: Predictive preloading working, visible reduction in pop-in during head turns.

Phase 3 — Simplified DB-LSTM in TF.js (Week 3–4)

This is the most ambitious phase, directly inspired by PredRNNv3.

Step 3.1 — Prepare training data From your collected gaze logs, create supervised samples: input = last 10 ticks of (x, y, vx, vy), output = (x, y) at tick +2 (200 ms ahead). Split 80/20 train/test.

Step 3.2 — Build the model in Python/PyTorch Implement a lightweight dual-branch architecture:

- Conv1D branch: 1D convolution over the time axis — captures local motion patterns (micro-saccades, velocity bursts)
- Attention branch: scaled dot-product self-attention over the 10-tick sequence — captures longer viewing pattern context
- DBMM-style gate: concatenate branch outputs, apply sigmoid gate + 1×1 linear to fuse them
- Output layer: Dense → (x, y) prediction

Target ~50K parameters — small enough for fast TF.js inference.

Step 3.3 — Train and compare Train with AdamW, MSE loss. Compare prediction error (pixels) against the Kalman filter on the test set. Even a small improvement here is a valid research contribution.

Step 3.4 — Export to TF.js and run in browser

bash

```
tensorflowjs_converter --input_format=tf_saved_model ./saved_model ./tfjs_model
```

Load in the browser with tf.loadLayersModel(). Run inference every tick — target under 5 ms. Swap it in as the new prediction source for the texture loader.

Milestone: ML model running live in browser, outperforming Kalman on held-out gaze sequences.

Phase 4 — Metrics Dashboard + A/B Evaluation (Week 5)

Step 4.1 — Instrument four metrics

- Pop-in events: increment a counter each frame a texture is needed but unavailable at the correct mip
- Load latency: performance.now() from request → load complete
- Memory usage: track total bytes currently held in GPU for textures
- FPS: existing from the demo, keep monitoring

Step 4.2 — A/B toggle Add a toggle in the UI that switches between reactive (current gaze) and predictive (Kalman or DB-LSTM) modes without reloading. Run each mode for 2 minutes, log all four metrics, export for comparison.

Step 4.3 — On-screen overlay Display the four metrics live on the canvas — similar to what the existing demo already does (FPS, detection %, jitter, error). Add the new streaming metrics to this overlay.

Milestone: Side-by-side quantitative proof that prediction reduces pop-in and load latency.

Phase 5 — Report + Stretch Goals (Week 6)

Report content:

- Model architecture (reference PredRNNv3's dual-branch design as inspiration)
- Quantitative results across all four metrics
- Ablation: reactive vs Kalman vs DB-LSTM
- Qualitative analysis (screen recording of visible pop-in reduction)

Stretch goal 1: Fuse head orientation (device gyroscope if on mobile) with eye gaze as two separate input streams — analogous to PredRNNv3's dual memory paths.

Stretch goal 2: Semantic prioritization — classify texture objects by type (UI element, interactive, background) and preload in priority order regardless of distance from predicted gaze.

Full PredRNNv3 is a PyTorch model trained on GPU — you can't run that natively in the browser. But you can implement a **faithful lightweight version** using **TensorFlow.js**, which runs inference entirely in-browser on WebGL. Here's the mapping:

PredRNNv3 Component	Browser Equivalent
CNN branch (3×3 conv + channel attention)	tf.layers.conv2d on gaze heatmap patches
Transformer branch (MWSA + SwiGLU)	Simplified self-attention via tf.matMul
DB-LSTM memory cells (C and M)	Two separate LSTM state tensors in TF.js

PredRNNv3 Component	Browser Equivalent
DBMM fusion	1×1 conv + sigmoid gate in TF.js
Training	Pre-train offline in Python/PyTorch → export to TF.js format → load in browser

---

Practical Implementation Plan

Step 1 — Data Collection (Build a Gaze Logger)

Before any model, you need training data. Extend the existing demo to log gaze sequences:

javascript

```
// In trackFace(), add:

gazeLog.push({ t: Date.now(), x: gazeX, y: gazeY });

// Keep a rolling 5-second buffer, export as JSON
```

Collect 20–30 minutes of gaze data from real users watching videos. Each sample is a sequence of (x, y) coordinates at 10 Hz.

Step 2 — Offline Training (Python/PyTorch)

This is where PredRNNv3's architecture comes in. You'd implement a **scaled-down DB-LSTM** in Python:

python

```
# Input: sequence of (x, y, vx, vy) — position + velocity

# at T=10 timesteps

# Output: predicted (x, y) at T+2 (200ms ahead)


# CNN branch: 1D conv over the time dimension

# Transformer branch: self-attention over the sequence

# DBMM: gate-fused output
```

The key simplification vs. the paper: your input is a 1D sequence of gaze coordinates (not 2D video frames), so the CNN becomes a **Conv1D** and the Transformer operates over timesteps rather than spatial patches. This is much lighter — the paper uses 6.2M parameters, your version might need only ~50K.

Then export to TF.js:

bash

```
tensorflowjs_converter --input_format=tf_saved_model ./model ./tfjs_model
```

Step 3 — In-Browser Inference

Load the model and run prediction each tick:

javascript

```
const model = await tf.loadLayersModel('./tfjs_model/model.json');


// In trackFace(), after computing gazeX/gazeY:

const inputTensor = tf.tensor3d([gazeBuffer]); // shape [1, 10, 4]

const predicted = model.predict(inputTensor);   // shape [1, 2]

const [predX, predY] = await predicted.data();


// Use predX/predY for texture preloading instead of current gazeX/gazeY
```

The inference latency for a 50K-parameter model in TF.js is typically under 5ms — well within your 100ms tick budget.

Step 4 — Wire Prediction Into Texture Streaming

Once you have predicted gaze coords 200ms ahead, the texture streaming manager preloads high-res patches **at the predicted location** rather than the current one. This is the core project contribution — the prediction model feeds directly into streaming decisions.

Honest Assessment of Complexity

Task	Difficulty	Time
Gaze data logger	Easy	1 day
Simplified DB-LSTM in PyTorch	Medium	3–4 days
TF.js export + browser inference	Easy-Medium	1–2 days
Three.js scene + texture streaming	Medium	3–4 days
End-to-end integration + eval	Medium	2–3 days

The paper's full architecture (ViT with MWSA, SwiGLU FFN, DBMM) is complex but the **ideas** are what matter — even a simplified 2-branch LSTM with a basic attention mechanism over your gaze sequence would be a strong project contribution and directly inspired by PredRNNv3.

Suggested Positioning for Your Report

You don't need to implement PredRNNv3 exactly. The framing would be:

"Inspired by PredRNNv3's dual-branch architecture for spatiotemporal prediction, we implement a lightweight browser-compatible variant that separates local gaze dynamics (CNN branch) from global viewing pattern modeling (attention branch), fused via a modulation gate — adapted for 1D gaze sequences and deployed via TensorFlow.js for real-time inference."

10. References

- Guo, G. & Zhou, Y. (2026). PredRNNv3: A CNN-transformer collaborative recurrent neural network for spatiotemporal prediction. Neurocomputing, 664, 132078. https://doi.org/10.1016/j.neucom.2025.132078
- Wang, Y. et al. (2022). PredRNN: A recurrent neural network for spatiotemporal predictive learning. IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(2), 2208–2225.
- Tang, S. et al. (2023). SwinLSTM: Improving spatiotemporal prediction accuracy using Swin Transformer and LSTM. ICCV 2023, 13470–13479.
- face-api.js (v0.22.2). Vincent Mühler. https://github.com/justadudewhohacks/face-api.js
- Three.js (r157). Mr. doob et al. https://threejs.org
- TensorFlow.js (4.x). Google LLC. https://www.tensorflow.org/js